Quiz — 2.3.1 Algorithms

Name: ____ Date: _ ***Mark:*** ___ / 35

OCR H446 · Component 02 · 2.3.1 — show your traces

Section A — Multiple choice (10 marks)

A1. What is the worst-case time complexity of **binary search**? A. O(1) B. O(log n) C. O(n) D. O(n log n) Your answer: ☐

A2. What is the **average** time complexity of **merge sort**? A. O(n) B. O(n log n) C. $O(n^2)$ D. $O(2^n)$ Your answer: ☐

A3. What is the **worst-case** time complexity of **quick sort**? A. O(log n) B. O(n log n) C. $O(n^2)$ D. $O(2^n)$ Your answer: ☐

A4. A list must be in a particular state before **binary search** can be used. What is it? A. The list must be sorted B. The list must contain only integers C. The list must have an even number of items D. The list must be stored in a linked list Your answer: ☐

A5. Which data structure does a **breadth-first traversal** use to keep track of nodes still to visit? A. Stack B. Queue C. Binary tree D. Hash table Your answer: ☐

A6. Simplify the expression `4n² + 200n + 9` to Big O notation. A. O(n) B. O(200n) C. $O(n^2)$ D. $O(4n^2)$ Your answer: ☐

A7. What is the time complexity of **pushing** an item onto a **stack**? A. O(1) B. O(log n) C. O(n) D. $O(n^2)$ Your answer: ☐

A8. Which traversal of a **binary search tree** outputs the values in **ascending order**? A. Pre-order B. In-order C. Post-order D. Breadth-first Your answer: ☐

A9. A problem that can only be solved in **exponential time or worse** is described as: A. Tractable B. Intractable C. Logarithmic D. Stable Your answer: ☐

A10. What is the key difference between **A*** and **Dijkstra's** algorithm? A. A *can handle negative edge weights B. A* uses a heuristic estimate of the remaining cost to the goal C. A *explores every node in the graph; Dijkstra does not D. A* does not guarantee finding a path Your answer: ☐

Section B — Traces & short answer (20 marks)

B1. A program contains the following nested loop structure over a list of `n` items:

```
for i in range(n):
    for j in range(n):
        compare items
```

State the Big O time complexity and **justify** your answer. **[3]**

....

....

....

B2. Trace a **bubble sort** (one pass = comparing adjacent pairs left-to-right, swapping if out of order) on the list below. Show the list **after each complete pass**. **[4]**

Start: `[5, 3, 8, 1, 4]`

After pass	Item 1	Item 2	Item 3	Item 4	Item 5
Pass 1					
Pass 2					
Pass 3					
Pass 4					

B3. A **binary search** is performed on the sorted list below to find the value **23**.

`[4, 9, 15, 18, 23, 27, 31, 42, 56]` (indices 0–8)

Complete the trace table. Show each step until the value is found. **[4]**

Step	low	high	mid (index)	value at mid	< = > target?
1					
2					
3					

State the result: ....

B4. Consider the binary tree below.

```
          40
         /  \
       25    60
      /  \   /  \
    15   30 50   70
```

Write the output of each traversal. **[3]**

(a) **Pre-order:** .... **[1]**

(b) **In-order:** .... **[1]**

(c) **Post-order:** .... **[1]**

B5. State which traversal type (DFS or BFS) uses a **stack** and which uses a **queue**, and give **one** practical use of a breadth-first traversal. **[3]**

....

....

....

B6. Dijkstra's algorithm is run on the weighted graph below, starting at node **A**, to find the shortest distance to every other node.

```
       4        2
   A ------ B ------ C
   |        |        |
  1|       5|       3|
   |        |        |
   D ------ E ------ F
       2        1
```

Edges (undirected, with weights): A–B = 4, A–D = 1, B–C = 2, B–E = 5, C–F = 3, D–E = 2, E–F = 1

Complete the table of **final shortest distances** from A. **[3]**

Node	Shortest distance from A	Via (previous node)
A	0	—
B		
C		
D		
E		
F		

Section C — Exam-style question (5 marks)

C1. A games developer needs an algorithm to find the shortest route for a non-player character (NPC) to travel from its current position to a **single, known target location** on a large map.

Compare the suitability of **Dijkstra's algorithm** and the **A*** algorithm for this task, and justify which you would recommend. **[5]**

....

....

....

....

....

....

....

....